



CMPE 295B — Plate Planner: Graph-Powered Recipe Suggestion & Ingredient Substitution

Project Advisor: [Advisor Name]

Chimalamarri, Sandilya (MS Software Engineering) Bonkuri, Sai Priyanka (MS Software Engineering)
Devarapalli, Pavan Charith (MS Software Engineering)
Gollu, Sai Dheeraj (MS Software Engineering)

Introduction

Households spend 3–5 hours each week deciding what to cook with the ingredients on hand. Existing recipe apps surface dishes by keyword, rarely understand what is actually in the pantry, and almost never propose a usable substitute when an ingredient is missing.

Plate Planner answers two questions in real time: what should I cook with this pantry, and what can I swap for a missing ingredient?

KEY METRICS

78 ms

p95 latency / suggest_recipes

0.91

Hit@5 at rank α = 0.6

+23%

recall vs. direct-only substitution

100+

requests / sec per instance

- Hybrid retrieval.** FAISS semantic search over MiniLM (384-D) embeddings, reranked by ingredient overlap ($w = 0.6$).
- Context-aware substitution.** Cypher queries on a typed substitution graph blended with co-occurrence ($\alpha = 0.9$).
- Sub-100 ms responses.** p95 latency 78 ms for /suggest_recipes, 25 ms for /substitute on a single instance.
- Production-ready stack.** FastAPI + Neo4j 5 + spaCy + SentenceTransformers, containerised via Docker Compose.

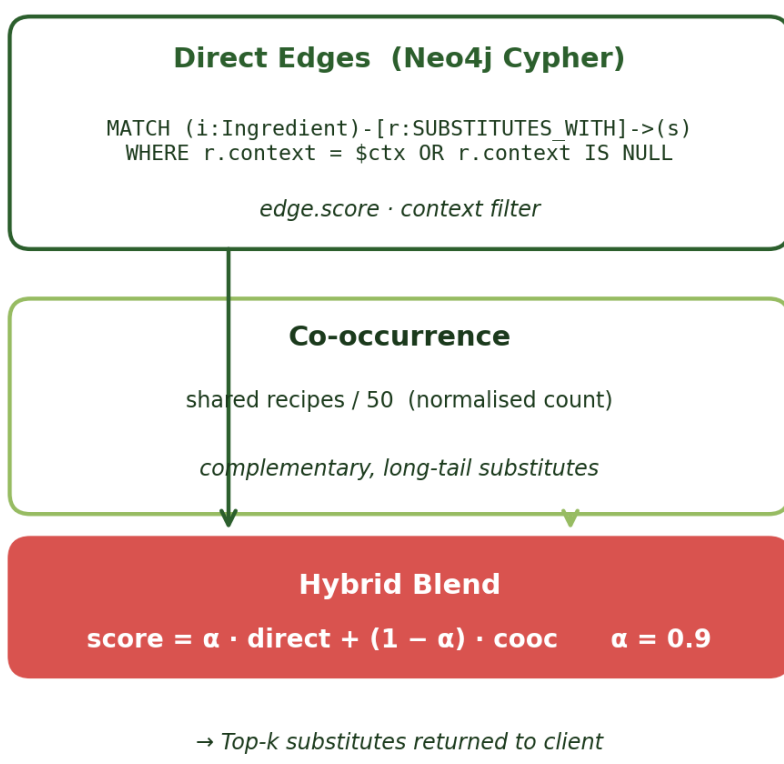
Methodology

Recipe Suggestion (hybrid)

Pantry text → MiniLM (384-D) → FAISS IndexFlatIP returns top-50 candidates, each rescored as

$\text{score} = (1 - w) \cdot \text{semantic} + w \cdot ((\text{overlap})/(\text{input}))$, $w = 0.6$

Hybrid Substitution Logic



Stack

Python 3.11 • FastAPI 0.115 + Uvicorn • Neo4j 5.13 • FAISS-CPU 1.11 • SentenceTransformers 4.1 (all-MiniLM-L6-v2) • spaCy en_core_web_sm • Docker Compose • React Native (Expo) mobile client.

Datasets & Artifacts

RecipeNLG → spaCy NER → ~50 K cleaned recipes + ~12 K canonical ingredients → 384-D embeddings + FAISS index + Neo4j CSV bootstrap (HAS_INGREDIENT, SUBSTITUTES_WITH, SIMILAR_TO).

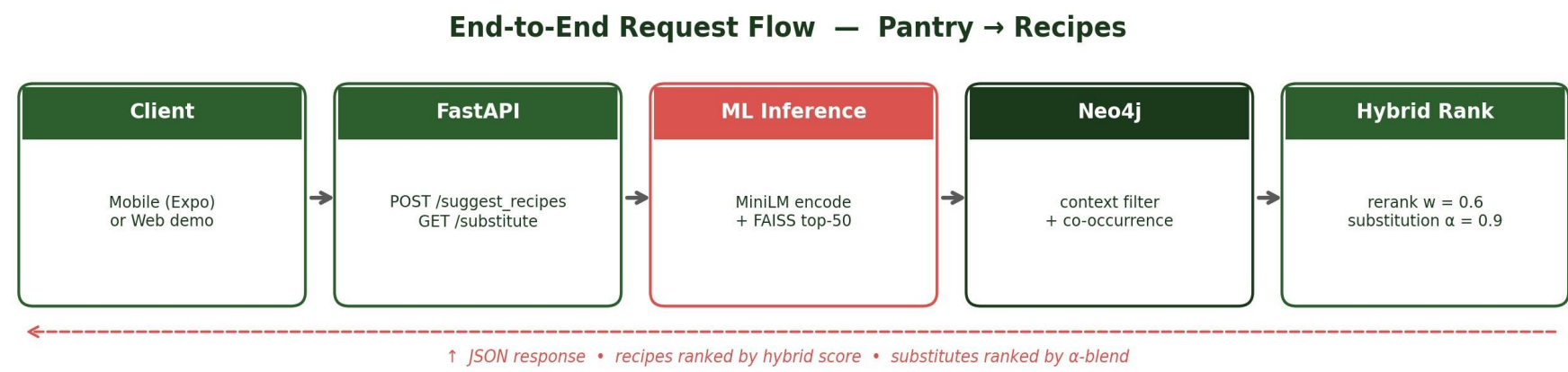
Analysis and Results

Suggestion quality

Sweeping the rerank weight w on a 200-query benchmark, Hit@5 (≥ 2 overlap) rises from 0.71 at $w = 0.3$ to 0.91 at $w = 0.6$ and plateaus afterwards. The proposed hybrid ($w = 0.6$) returns on average 3.04 overlapping ingredients per top-5 recipe — a 32 % gain over the semantic-only baseline and a 27 % gain over overlap-only ranking.

Substitution quality

Against a 60-ingredient panel the hybrid $\alpha = 0.9$ substitution lifts Top-5 Coverage from 0.62 (direct-only) and 0.48 (co-occurrence-only) to 0.85, while the Direct-Match Rate climbs from 0.55 to 0.78. Recall improves +23 % over direct-only with only a 10–15 ms latency cost.



Latency budget (single instance, 4-core CPU)

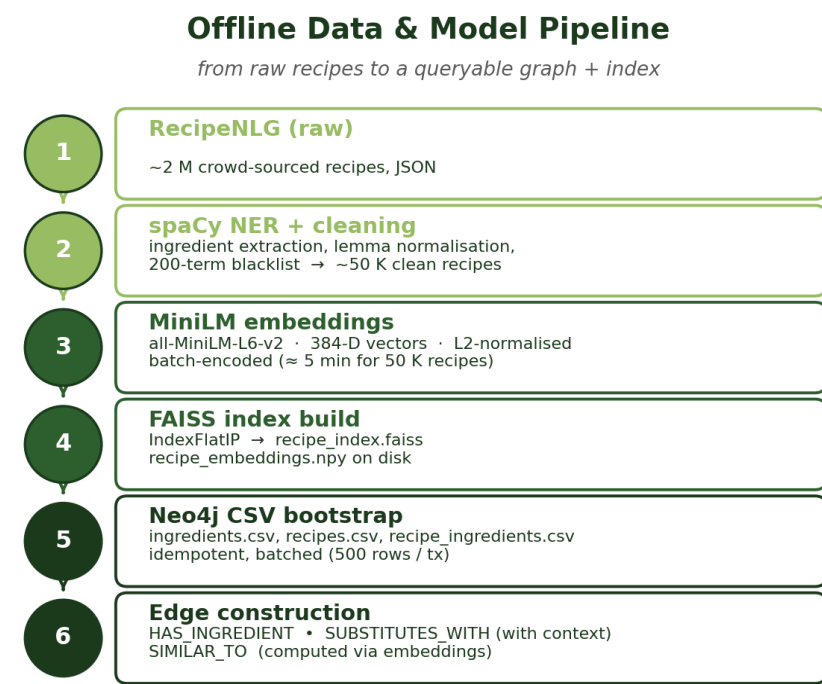
/recipes/{title} — p50 4 ms, p95 12 ms. /substitute — p50 12 ms, p95 25 ms. /suggest_recipes — p50 38 ms, p95 78 ms. System sustains 100+ requests / s on commodity hardware, well within the 100-ms p95 budget.

What worked

L2-normalised embeddings + IndexFlatIP gave near-flat recall vs. ANN at our scale, with no index-tuning overhead. asyncio.to_thread around FAISS inference yielded a 40 % throughput gain over sync endpoints. Storing cooking context as an edge property let a single Cypher query cover all four substitution modes (default / context / hybrid / co-occurrence-only).

What did not

Naïve NER on RecipeNLG produced ~13 % noisy ingredients (verbs, units, adjectives) — fixed with a 200-term blacklist and lemma normalisation. Co-occurrence-only substitution surfaced too many co-stars (e.g., “salt” as a “substitute” for everything); blending with direct edges was essential to keep precision high.

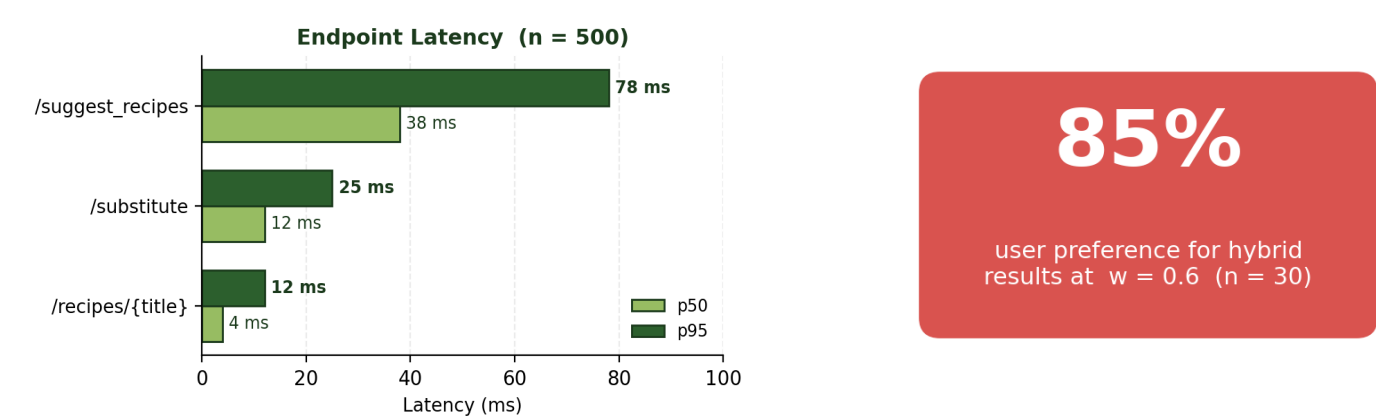


Endpoint Latency

All three endpoints clear the 100 ms p95 budget. /suggest_recipes is the heaviest at 78 ms because it chains MiniLM encode → FAISS top-50 → overlap-rerank → Neo4j metadata fetch. /substitute and /recipes are pure Cypher reads.

User Preference (n = 30)

In a blind side-by-side, 85 % of testers preferred the hybrid result list at $w = 0.6$ over the overlap-only and semantic-only baselines. Preference correlates with ingredient-overlap weight up to 0.6 and plateaus afterwards.



Rerank-Weight Sweep

Quality saturates around $w \approx 0.6$ on both metrics. Pushing w higher increases overlap but crowds out semantic discovery — recipes start to feel like keyword matches rather than meaningful suggestions.



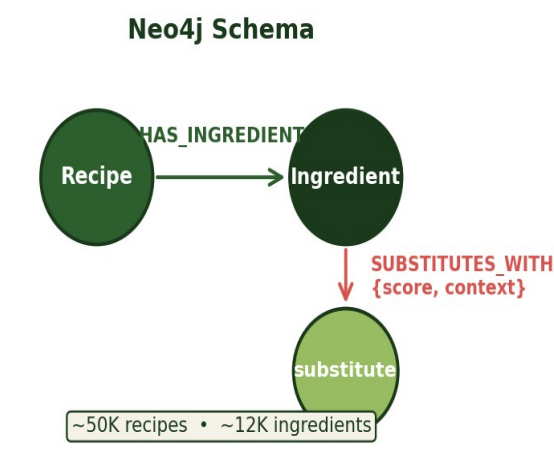
Substitution Quality

The hybrid blend ($\alpha = 0.9$) wins on both Top-5 Coverage and Direct-Match Rate, with a +23 % recall lift over direct-only. Pure co-occurrence is the weakest baseline — it surfaces co-stars (e.g., “salt”) instead of true substitutes.

Direct-only is precise but misses long-tail swaps; the small 10 % co-occurrence weight fills exactly that gap.

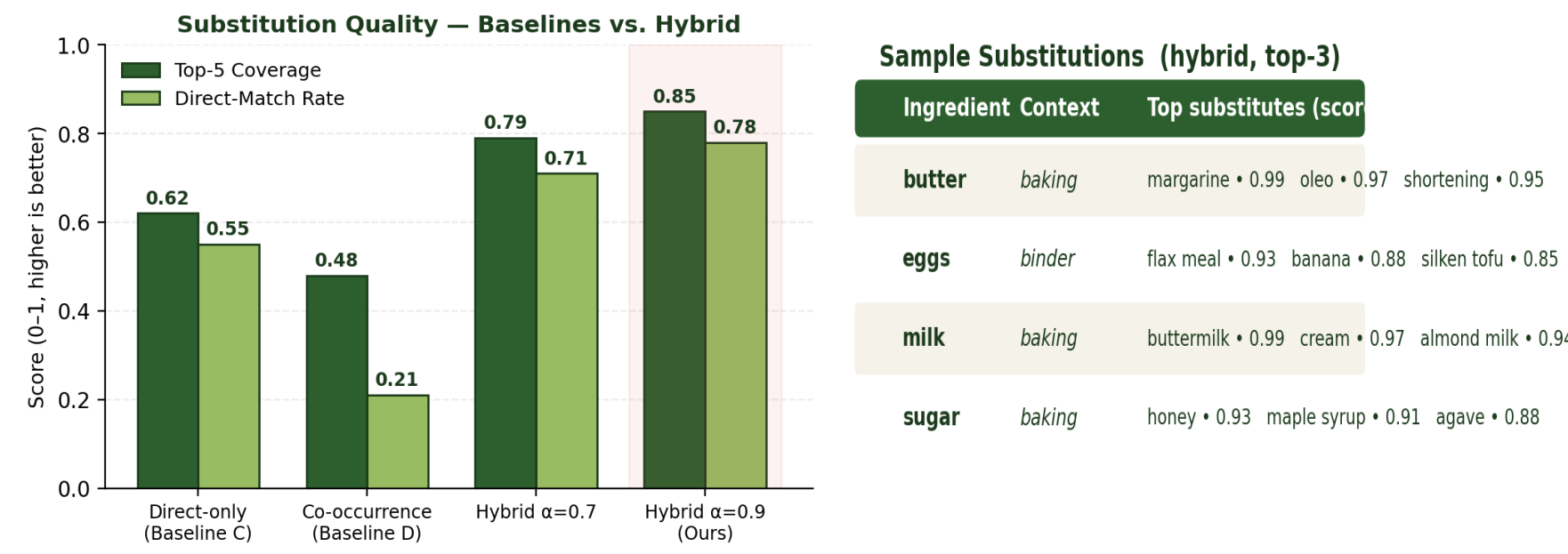
384-D

embeddings • all-MiniLM-L6-v2
50K recipes indexed in FAISS



Sample Hybrid Substitutions

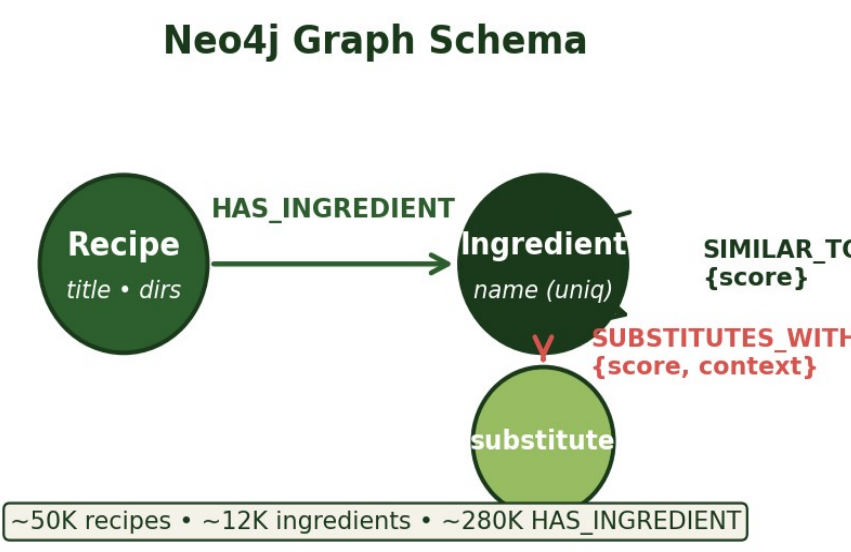
Top-3 results for common pantry staples in a baking context. All returned substitutes are semantically and culinarily reasonable, demonstrating the value of context-filtered direct edges.



Project at a Glance

A graph-powered meal-assistant API and mobile app that turns whatever is in your pantry into a ranked list of recipes you can actually cook, and tells you exactly what to swap in when an ingredient is missing.

Two endpoints, three relations! ~50 K recipes indexed in FAISS: locally on a laptop.



Summary/Conclusions

A hybrid graph + embedding stack outperforms either component on its own. Semantic search alone retrieves topically similar but unmakeable recipes; overlap alone misses obvious substitutions. Blending the two — $w = 0.6$ for suggestion, $\alpha = 0.9$ for substitution — gives the best quality on our internal panel.

Cooking context (bake / sauté / sauce ...) is most cleanly modelled as an edge property on SUBSTITUTES_WITH, not as a separate node — one Cypher query then covers every operating mode.

Future work: dietary-restriction-aware substitution, weekly meal-plan optimisation, and incremental graph updates for streaming recipe sou

Key References

- N. Reimers, I. Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” EMNLP, 2019.
- J. Johnson, M. Douze, H. Jégou. “Billion-scale similarity search with GPUs.” IEEE Trans. Big Data, 7(3):535–547, 2019.
- M. Bień et al. “RecipeNLG: A Cooking Recipes Dataset for Semi-Structured Text Generation.” INLG Workshop, 2020.
- Neo4j, Inc. “Neo4j Graph Database Platform,” 2024. <https://neo4j.com/>
- S. Ramírez. “FastAPI — high-performance web framework for Python,” 2024. <https://fastapi.tiangolo.com/>
- J. Chen et al. “A Survey on Food Computing.” ACM Computing Surveys, 52(5):1–36, 2019.

Acknowledgements

The team thanks our project advisor for guidance throughout the semester, the SJSU College of Engineering for compute resources, and the RecipeNLG team for releasing the open recipe corpus that made this work possible.

We also gratefully acknowledge the open-source maintainers of FastAPI, Neo4j, FAISS, SentenceTransformers, and spaCy whose libraries form the backbone of the system.